



Bachelor-Thesis

Entwicklung und Evaluation agentenbasierter Systeme im
Software Engineering: Ein Ansatz zur Automatisierung von
Entwicklungsprozessen mittels Large Language Models

zur Erlangung des akademischen Grades

Bachelor of Science

im dualen Studiengang Informatik an der IU Internationale Hochschule

von

Maria Musterfrau

Matrikelnummer: 1234567

Musterstraße 1, 12345 Musterstadt

12. April 2026

Referent: Prof. Dr. Klaus Quibeldey-Cirkel

Korreferent: Prof. Dr. Philipp Diebold

Kurzfassung

Kontext und Motivation: Large Language Models (LLMs) entwickeln sich zunehmend von reinen Textgeneratoren zu agentischen Systemen, die planen, Werkzeuge einbinden und mehrschrittige Aufgaben bearbeiten können. Für das Software Engineering eröffnet dies neue Möglichkeiten zur Unterstützung von Code-Review, Testgenerierung, Fehlersuche und Refactoring.

Zielsetzung: Die Arbeit untersucht die Entwicklung und Evaluation agentischer Architekturen für Software-Engineering-Aufgaben. Im Zentrum stehen drei Fragen: (1) Wie lassen sich robuste Agentenstrategien für SE-Workflows systematisch modellieren? (2) Welche Architekturprinzipien ermöglichen eine sichere und nachvollziehbare Werkzeugintegration? (3) Mit welchen Metriken kann die Leistungsfähigkeit agentischer Systeme unter realistischen Randbedingungen bewertet werden?

Methodik: Auf Basis einer strukturierten Literaturanalyse wird eine Referenzarchitektur entwickelt, die Planung, Werkzeugnutzung, episodisches Gedächtnis und Sicherheitsmechanismen integriert. Darauf aufbauend wird ein prototypisches System in Python implementiert und anhand reproduzierbarer Szenarien aus dem Software Engineering evaluiert.

Ergebnisse: Die Evaluation zeigt für das gewählte Szenarienset, dass die entwickelte Architektur bei automatisiertem Refactoring eine Erfolgsrate von 73 % erreicht und die mittlere Bearbeitungszeit um 45 % reduziert. Reflexionsmechanismen verbessern die Robustheit gegenüber fehlerhaften Werkzeugausgaben um 28 %; optimierte Kontextverwaltung senkt die Token-Nutzung um bis zu 40 %. Die Resultate sind als indikative Ergebnisse einer prototypischen Umsetzung zu interpretieren und nicht als unmittelbar auf öffentliche Leaderboards übertragbare Vergleichswerte.

Beitrag: Die Arbeit liefert eine praxisnahe Referenzarchitektur, konkrete Implementierungsrichtlinien und ein mehrdimensionales Evaluationsschema für agentische SE-Systeme. Sie zeigt technische Potenziale, benennt methodische Grenzen und ordnet die Ergebnisse in die aktuelle Forschung zu agentischer Softwareentwicklung ein.

Abstract

Context and Motivation: Large Language Models (LLMs) are increasingly evolving from pure text generators into agentic systems that can plan, invoke tools, and handle multi-step tasks. In software engineering, this creates new opportunities to support code review, test generation, debugging, and refactoring.

Objective: This thesis investigates the development and evaluation of agentic architectures for software engineering tasks. The central questions are: (1) How can robust agent strategies for SE workflows be modeled systematically? (2) Which architectural principles enable secure and traceable tool integration? (3) Which metrics allow a realistic assessment of agentic systems under practical constraints?

Methodology: Based on a structured literature review, a reference architecture is developed that integrates planning, tool usage, episodic memory, and safety mechanisms. A prototypical system is implemented in Python and evaluated using reproducible software-engineering scenarios.

Results: For the selected evaluation scenarios, the proposed architecture achieves a success rate of 73 % in automated refactoring and reduces mean processing time by 45 %. Reflection mechanisms improve robustness against faulty tool outputs by 28 %, while optimized context management reduces token usage by up to 40 %. These values should be interpreted as indicative prototype results rather than as directly comparable public leaderboard scores.

Contribution: The thesis provides a practical reference architecture, concrete implementation guidelines, and a multidimensional evaluation scheme for agentic SE systems. It identifies technical opportunities, names methodological limitations, and situates the findings within the current research landscape on agentic software engineering.

Danksagung

An dieser Stelle möchte ich meinen aufrichtigen Dank aussprechen.

Zuallererst danke ich meinen Betreuern **Prof. Dr. Klaus Quibeldey-Cirkel** und **Prof. Dr. Philipp Diebold** für die fachliche Unterstützung, wertvollen Hinweise und die Möglichkeit, die Arbeit zu verfassen. Ihre offene und konstruktive Kritik hat maßgeblich zur Qualität der Arbeit beigetragen.

Mein besonderer Dank gilt auch meinen Kommilitoninnen und Kommilitonen, die durch fachliche Diskussionen und Feedback meine Gedanken geschärft haben. Ihre Perspektiven haben mir geholfen, verschiedene Aspekte der Thematik tiefergehend zu beleuchten.

Darüber hinaus möchte ich denjenigen danken, die mir während des Schreibprozesses moralische Unterstützung gegeben haben. Ihre Geduld und Ermutigung waren unverzichtbar für den Erfolg der Arbeit.

Abschließend danke ich den Entwicklern und der Open-Source-Community für die Bereitstellung der vielfältigen Werkzeuge und Bibliotheken, ohne die die Arbeit nicht möglich gewesen wäre.

Hinweis zur geschlechtergerechten Sprache

In der Arbeit wird auf eine geschlechtergerechte Sprache geachtet. Aus Gründen der besseren Lesbarkeit wird in der Arbeit eine Mischung verschiedener Formen geschlechtergerechter Sprache verwendet:

- **Geschlechtsneutrale Formulierungen** (z. B. Studierende, Mitarbeitende, Person)
- **Paarformen** (z. B. Entwicklerinnen und Entwickler, Nutzerinnen und Nutzer)
- **Genderstern** (z. B. Benutzer*innen) zur Sichtbarmachung aller Geschlechter

Alle verwendeten Personenbezeichnungen gelten gleichermaßen für alle Geschlechter. Die Arbeit orientiert sich an den Empfehlungen des Leitfadens „Sag’s doch gleich! Geschlechtergerechte Sprache an Thüringer Hochschulen“ des Thüringer Kompetenznetzwerks Gleichstellung (TKG).

Inhaltsverzeichnis

Kurzfassung	i
Abstract	iii
Danksagung	v
Hinweis zur geschlechtergerechten Sprache	vii
Abbildungsverzeichnis	xiii
Tabellenverzeichnis	xv
Listings	xvii
Abkürzungsverzeichnis	xix
1 Einführung	1
1.1 Motivation und Relevanz	1
1.2 Problemstellung	2
1.3 Zielsetzung	2
1.4 Abgrenzung des Themas	3
1.5 Aufbau der Arbeit	4
2 Theoretischer Hintergrund	5
2.1 Grundkonzepte	5
2.1.1 Large Language Models: Grundlagen	5
2.1.2 Von LLMs zu Agenten: Erweiterte Automatisierungslogik . .	5
2.1.3 Agentic AI: Begriffe und Bausteine	6
2.1.4 Chain-of-Thought und Reasoning-Patterns	6
2.1.5 Tool-Nutzung in LLMs	7
2.1.6 Etablierte Frameworks und Plattformen	7
2.2 Verwandte Arbeiten	8
2.2.1 Reasoning-and-Acting-Patterns	8

2.2.2	Software Engineering Agents	9
2.2.3	Multi-Agent-Systeme	9
2.2.4	Graph-/Workflow-basierte Orchestrierung	10
2.2.5	Evaluations-Benchmarks	10
2.3	Vergleich und Bewertung	11
2.4	Forschungslücke	12
2.4.1	Architektonische Lücken	12
2.4.2	Evaluations-Herausforderungen	12
2.4.3	Skalierungs- und Kostenfragen	12
2.4.4	Beitrag der Arbeit	13
3	Konzept und Methodik	15
3.1	Übersicht des Lösungsansatzes	15
3.2	Architektur und Design	15
3.2.1	Designprinzipien	16
3.2.2	Architekturkomponenten	16
3.2.3	ReAct-Loop im Detail	17
3.2.4	Tool-Interface-Design	17
3.3	Methodik	18
3.3.1	Entwicklungsmethodik	18
3.3.2	Evaluationsmethodik	19
3.3.3	Testszenarien	19
3.3.4	Fallbeispiel: Refactoring-Flow	21
3.4	Sicherheitsmechanismen	22
3.4.1	Input-Validation	22
3.4.2	Sandboxing	22
3.4.3	Audit-Logging	22
3.4.4	Menschliche Freigabe	23
3.5	Abgrenzung zu alternativen Ansätzen	23
4	Implementierung und Ergebnisse	25
4.1	Implementierungsdetails	25
4.1.1	Technologiestack	25
4.1.2	Projektstruktur	25
4.1.3	Kernimplementierung: Agent-Controller	26

4.2	Experimentelles Setup	29
4.2.1	Testumgebung	29
4.2.2	Benchmark-Szenarien	29
4.2.3	Beispiel: Test Failure Diagnosis — Schritt-für-Schritt	30
4.3	Ergebnisse	31
4.3.1	Erfolgsraten nach Aufgabentyp	31
4.3.2	Effizienzmetriken	32
4.3.3	Qualitätsmetriken	32
4.3.4	Robustheit und Fehlerbehandlung	32
4.4	Vergleich mit existierenden Ansätzen	33
4.5	Validierung und Verifikation	34
4.5.1	Unit-Tests	34
4.5.2	Integrationstests	34
4.5.3	Sicherheits-Audits	34
5	Fazit und Ausblick	37
5.1	Zusammenfassung der Ergebnisse	37
5.1.1	Beantwortung der Forschungsfragen	37
5.1.2	Empirische Validierung	38
5.2	Beiträge der Arbeit	39
5.2.1	Wissenschaftlicher Beitrag	40
5.3	Limitierungen	40
5.3.1	Methodische Limitierungen	40
5.3.2	Technische Limitierungen	41
5.3.3	Gesellschaftliche und ethische Limitierungen	41
5.4	Zukünftige Arbeiten	42
5.4.1	Kurzfristige Erweiterungen	42
5.4.2	Mittel- bis langfristige Forschungsrichtungen	42
5.4.3	Offene Forschungsfragen	43
5.5	Schlusswort	43
	Literaturverzeichnis	47
	Index	50
A	Verzeichnis der KI-Nutzung	55

Abbildungsverzeichnis

3.1	Agentenarchitektur für Software Engineering mit agentischer KI (TikZ-Diagramm)	17
3.2	ReAct-Workflow: Zyklisches Paradigma aus Denken und Handeln mit Fehlertoleranz	20

Tabellenverzeichnis

2.1	Vergleich agentischer Systemtypen nach Fähigkeiten und Anwendungsbereich	11
4.1	Vergleich mit Baseline-Systemen (auf gleichem Benchmark-Set) . .	33
4.2	Detaillierte Benchmark-Ergebnisse: Agentische SE-Szenarien mit Evaluationsmetriken	35
4.3	Evaluationsmetriken für agentische SE-Workflows (Beispieltabelle) .	35
A.1	Verzeichnis der in dieser Arbeit genutzten KI-Werkzeuge	56

Listings

3.1	Tool-Interface-Schema (Pseudocode)	17
3.2	Human-Approval-Mechanismus (Pseudocode)	23
4.1	Minimaler agentischer Loop für SE-Aufgaben (Beispiel-Listing)	26
4.2	Werkzeugaufruf-Stub in TypeScript mit einfachem Funktionsschema (Beispiel-Listing)	27

Abkürzungsverzeichnis

Abkürzung	Bedeutung
ACI	Agent-Computer-Interface
AI	Artificial Intelligence (Künstliche Intelligenz)
API	Application Programming Interface (Anwendungsprogrammierschnittstelle)
APPS	Automated Programming Progress Standard
AST	Abstract Syntax Tree (Abstrakter Syntaxbaum)
CD	Continuous Delivery (Kontinuierliche Auslieferung)
CI	Continuous Integration
CLI	Command Line Interface (Befehlszeilenschnittstelle)
CoT	Chain-of-Thought (Gedankenkette)
DB	Datenbank (Database)
DFG	Deutsche Forschungsgemeinschaft
E/A	Eingabe/Ausgabe (Input/Output)
FAQ	Frequently Asked Questions (Häufig gestellte Fragen)
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit (Grafik-Verarbeitungseinheit)
HTTP	HyperText Transfer Protocol
I/O	Input/Output (Eingabe/Ausgabe)
Fortsetzung auf nächster Seite	

Abkürzung	Bedeutung
IDE	Integrated Development Environment (Integrierte Entwicklungsumgebung)
JSON	JavaScript Object Notation
KB	Knowledge Base (Wissensdatenbank)
KG	Knowledge Graph (Wissensgraph)
KI	Künstliche Intelligenz
LLM	Large Language Model (Großes Sprachmodell)
LOC	Lines of Code (Codezeilen)
MBPP	Mostly Basic Python Problems
ML	Machine Learning (Maschinelles Lernen)
NLP	Natural Language Processing (Natürlichsprachverarbeitung)
PII	Personally Identifiable Information (Personenbezogene Daten)
PR	Pull Request
PRD	Product Requirement Document (Produktanforderungsdokument)
QA	Quality Assurance (Qualitätssicherung)
QS	Qualitätssicherung
RAG	Retrieval-Augmented Generation (Abruf-gestützte Generierung)
RAM	Random Access Memory (Arbeitsspeicher)
ReAct	Reasoning and Acting (Denken und Handeln)

Fortsetzung auf nächster Seite

Abkürzung	Bedeutung
REST	Representational State Transfer
RFC	Request for Comments (IETF-Standarddokumente)
RLHF	Reinforcement Learning from Human Feedback
SE	Software Engineering (Software-Entwicklung)
SMT	Satisfiability Modulo Theories
SQL	Structured Query Language (Strukturierte Abfragesprache)
SSD	Solid State Drive (Festkörperspeicher)
SSH	Secure Shell
VCS	Version Control System (Versionskontrollsystem)
XML	Extensible Markup Language
YAML	YAML Ain't Markup Language

1 Einführung

„Wir erleben die Entstehung eines neuen Betriebssystems: Das LLM fungiert als Kernel-Prozess, das Kontextfenster als Arbeitsspeicher und externe Werkzeuge als E/A-Schnittstellen. Dies markiert einen fundamentalen Wandel in der Art und Weise, wie wir Computerbefehle komponieren und Softwarearchitekturen denken.“ ([Kar23])

— Andrej Karpathy, *Intro to Large Language Models*, 2023

1.1 Motivation und Relevanz

Im Software Engineering zeichnet sich derzeit eine Verschiebung der Automatisierungslogik ab: *agentische KI* – also KI-Systeme, die Ziele interpretieren, Pläne erstellen, Werkzeuge verwenden und Ergebnisse eigenständig prüfen – erweitert klassische Automatisierung um adaptive, mehrschrittige Problemlösung.¹

Darüber hinaus verlangt die praktische Anwendung agentischer Systeme einen belastbaren Ausgleich zwischen Automatisierung und menschlicher Kontrolle. In vielen Entwicklungsprozessen sind Schritte, die bislang überwiegend manuell erfolgen – etwa Code-Reviews, Testgenerierung oder Release-Checks – geeignete Kandidaten für assistierte Abläufe. Agenten können Routinearbeit übernehmen, erzeugen damit jedoch zugleich neue Anforderungen an Validierung, Protokollierung und Eingriffsmöglichkeiten. Die vorliegende Arbeit untersucht diesen Zielkonflikt und entwickelt Gestaltungsprinzipien für einen kontrollierten produktiven Einsatz. Wie Richards und Ford betonen, „müssen Architekturprinzipien auf Skalierbarkeit und Wartbarkeit ausgelegt sein“ ([RF20]), um praktischen Anforderungen gerecht zu werden. Für Forschung und Praxis ergeben sich daraus neue Architektur- und Methodikfragen: Wie lassen sich robuste agentische Workflows entwerfen? Wie werden Werkzeugnutzung,

¹ Der Begriff *agentische KI* bezeichnet Systeme, die über mehrere Schritte hinweg komplexe Aufgaben bearbeiten und dabei Planung, Werkzeugnutzung und Rückkopplung kombinieren.

Gedächtnis und Langkontext so orchestriert, dass nachvollziehbare Entscheidungen entstehen? Und wie lassen sich Sicherheit, Prüfbarkeit und Testbarkeit systematisch in agentische Systeme integrieren?²

1.2 Problemstellung

Vor dem Hintergrund adressiert die Arbeit exemplarisch die Entwicklung und Evaluation eines agentischen Systems für Softwareentwicklungsaufgaben (z. B. Refactoring, Code-Review, Generierung von Tests). Zentrale Fragen sind:

- Wie lassen sich Agentenstrategien (Planen, Werkzeugaufrufe, Selbstkritik) systematisch modellieren?
- Wie werden externe Werkzeuge (VCS, CI, Linter, Issue-Tracker) sicher und nachvollziehbar eingebunden?
- Welche Metriken messen Fortschritt, Qualität und Sicherheit realistisch?

Die hier formulierten Probleme sind sowohl konzeptioneller als auch praktischer Natur: Im Mittelpunkt stehen nicht nur die abstrakte Modellierung von Strategien, sondern ebenso konkrete Implementierungsfragen (Schnittstellen, Serialisierung, Fehlerbehandlung) und Fragen des Evaluationsdesigns (Benchmarks, Messgrößen, Reproduzierbarkeit). Die Ergebnisse sollen Entwicklungsteams konkrete Handlungsempfehlungen geben, wie agentische Automatisierung schrittweise, kontrolliert und messtechnisch abgesichert eingeführt werden kann.

1.3 Zielsetzung

Die Ziele der Arbeit sind auf die Spezialisierung „Software Engineering mit agentischer KI“ zugeschnitten:³

1. Analyse des Forschungsstands zu agentischen Architekturen und Orchestrierungsframeworks

² Praktische Orchestrierung bezeichnet hier die Koordination von Planungsschritten, Werkzeugaufrufen und Gedächtniszugriffen in einer strukturierten Abfolge.

³ Siehe auch [Wan+23] für verwandte Arbeiten zu Agentenarchitekturen.

2. Entwurf einer referenzierbaren Agentenarchitektur für Software-Engineering-Aufgaben
3. Implementierung eines prototypischen Agenten mit Werkzeuganbindung und Gedächtnis
4. Evaluation anhand reproduzierbarer Benchmarks (Qualität, Kosten, Laufzeit, Sicherheit)

Kurz gefasst zielt die Arbeit darauf ab, aus Praxisproblemen generalisierbare Lösungen abzuleiten: Neben einem lauffähigen Prototyp steht die Frage im Vordergrund, welche Architekturmuster sich zuverlässig übertragen lassen und welche Messgrößen den Mehrwert gegenüber manuellen Prozessen belastbar quantifizieren. Damit richtet sich die Arbeit gleichermaßen an Forschende und an Verantwortliche in der Softwareentwicklung.

1.4 Abgrenzung des Themas

Die Arbeit fokussiert Agenten für Softwareentwicklungsaufgaben. Nicht im Fokus sind z. B. Reinforcement Learning from Human Feedback (RLHF) im Detail, Trainingsmethoden auf Rohdaten oder proprietäre Interna von Foundation Models. Ebenso werden Domänen außerhalb der Softwareentwicklung (z. B. Robotik) nicht betrachtet.

- Zu komplexe Spezialfälle, die für die Arbeit nicht relevant sind
- Historische Entwicklungen vor einem bestimmten Zeitpunkt
- Randbereiche, die außerhalb des Fokus liegen

Die Konzentration auf reproduzierbare Ergebnisse erlaubt es, bewusst auf tiefe Trainingsfragen zu verzichten; stattdessen wird die Interaktion mit bestehenden Foundation Models über APIs und Adapterlösungen untersucht. Diese Fokussierung erhöht die Nachvollziehbarkeit der Ergebnisse und verbessert die Anschlussfähigkeit an typische Software-Engineering-Umgebungen.

1.5 Aufbau der Arbeit

Der Aufbau der Arbeit folgt einem klaren Argumentationsgang: Zunächst werden in Kapitel 2 die relevanten Grundlagen und verwandten Arbeiten zusammengetragen. Darauf aufbauend beschreibt Kapitel 3 die entworfene Referenzarchitektur mit Fokus auf Zustandsmodellierung, Policy-Design und Schnittstellen. Kapitel 4 dokumentiert die Implementierung des Prototyps, zeigt zentrale Code-Beispiele und erläutert Integrationsdetails. Abschließend fasst Kapitel 5 die Ergebnisse zusammen, diskutiert Limitationen und gibt einen Ausblick auf weiterführende Forschungs- und Entwicklungsfragen.

Insgesamt soll die Arbeit nicht nur eine theoretische Diskussion bieten, sondern konkrete, übertragbare Architekturentscheidungen und Metriken bereitstellen, die in produktiven Entwicklungsumgebungen einsetzbar sind.

2 Theoretischer Hintergrund

2.1 Grundkonzepte

Das Kapitel führt in Grundbegriffe agentischer Systeme ein und bildet die theoretische Basis für Konzept und Implementierung.¹

2.1.1 Large Language Models: Grundlagen

Large Language Models (LLMs) sind neuronale Netze, die auf großen Textkorpora trainiert wurden und hochwertige Textgenerierung ermöglichen [Ope24; Tou+23]. Basierend auf der Transformer-Architektur [Vas+17] nutzen sie Selbstaufmerksamkeitsmechanismen, um kontextuelle Zusammenhänge über lange Sequenzen hinweg zu erfassen.

Aktuelle Modellfamilien der GPT-, Claude- und Llama-4-Klasse zeigen, dass sich sowohl Kontextlängen als auch Werkzeugnutzung und multimodale Fähigkeiten erweitert haben [Met25]. Dadurch können LLMs Aufgaben durch *Few-Shot-Learning* und *In-Context-Learning* häufig lösen, ohne explizit auf die konkrete Zielaufgabe nachtrainiert worden zu sein. Diese Eigenschaften bilden die Grundlage für agentische Anwendungen.

2.1.2 Von LLMs zu Agenten: Erweiterte Automatisierungslogik

Während klassische LLMs primär auf Textvervollständigung spezialisiert sind, zeichnen sich *agentische Systeme* durch erweiterte Fähigkeiten aus [Wan+23]. Zentral ist das **zielgerichtete Handeln**: Der Agent versteht übergeordnete Ziele und plant systematisch Schritte zur Zielerreichung. Die **Werkzeugnutzung** ermöglicht die Integration externer Tools wie APIs, Datenbanken und Code-Ausführung, wodurch die

¹ Die Grundbegriffe basieren auf etablierten Konzepten aus der KI-Forschung, werden aber im Kontext von Software Engineering neu interpretiert.

Fähigkeiten über reine Textgenerierung hinausgehen. Ein persistentes **Gedächtnis** speichert Kontextinformationen über mehrere Interaktionen hinweg und ermöglicht kontextbewusstes Arbeiten. Durch **Reflexion** erfolgt eine selbstkritische Bewertung eigener Outputs mit iterativer Verbesserung. Schließlich unterstützt **mehrschrittige Planung** die Dekomposition komplexer Aufgaben in ausführbare Teilschritte.

2.1.3 Agentic AI: Begriffe und Bausteine

Kernbausteine agentischer Systeme sind [Wen23; Yao+23]:

Zustand (State): Umfasst aktuellen Kontext, Ziele, Erinnerungen und Tool-Status. Der Zustand wird dynamisch aktualisiert.

Policy: Steuert Entscheidungsprozesse (Planung, Aktion, Selbstkritik). Kann regelbasiert oder LLM-gesteuert sein.

Werkzeuge (Tools): Externe Funktionen wie Code-Ausführung, Websuche, Dateizugriff, VCS-Operationen. Tools erweitern die Fähigkeiten des Agenten über reine Textgenerierung hinaus [Sch+23; Qin+24].

Gedächtnis (Memory): Speichert episodische (konkrete Ereignisse) und semantische (abstraktes Wissen) Informationen. Kann durch Vektor-Datenbanken oder strukturierte Speicher realisiert werden.

Orchestrierung koordiniert diese Komponenten durch Planung, Werkzeugauswahl, Fehlerbehandlung und Reflexion [Yao+23]. Typische Artefakte in SE-Workflows sind strukturierte Schnittstellen über **JSON** und **YAML**, Datenpersistenz mittels **SQL**-Datenbanken sowie sichere Remote-Operationen über **SSH**. Darüber hinaus beruhen viele Komponenten auf Methoden des maschinellen Lernens und der Sprachverarbeitung für Code- und Textverstehen; Wissensrepräsentation erfolgt etwa in Wissensdatenbanken und Wissensgraphen. API-Interaktionen erfolgen üblicherweise über **HTTP** und **REST**.

2.1.4 Chain-of-Thought und Reasoning-Patterns

Chain-of-Thought (CoT) Prompting [Wei+22] ermöglicht es LLMs, Zwischenschritte explizit zu formulieren. Dies kann die Qualität der Problembearbeitung verbessern:

„*Let’s think step by step*“ — Typischer CoT-Prompt, der schrittweises Denken anregt

Erweiterte Muster umfassen:

- **ReAct** (Reasoning + Acting): Kombiniert Gedankenketten mit Tool-Aufrufen [Yao+23]
- **Tree-of-Thought**: exploriert mehrere Begründungspfade parallel
- **Reflexion**: Selbstkritik und iterative Verbesserung [Shi+23]

2.1.5 Tool-Nutzung in LLMs

Die Fähigkeit von LLMs, externe Werkzeuge zu nutzen, ist für praktische Anwendungen zentral. *Toolformer* [Sch+23] zeigte, dass LLMs lernen können, wann und wie Tools aufzurufen sind. ToolLLM [Qin+24] erweiterte dies auf über 16.000 reale APIs.

Typischer Tool-Calling-Flow:

1. Agent identifiziert Bedarf für externes Tool
2. Generierung strukturierter Tool-Aufruf-Parameter (meist JSON)
3. Ausführung durch Host-System
4. Integration des Ergebnisses in Kontext
5. Fortsetzung der Aufgabe

2.1.6 Etablierte Frameworks und Plattformen

Praxisnahe Frameworks bieten Abstraktionen für agentische Systeme [Wu+23]:²

- **LangChain** [Cha22]: modulare Komponenten für Ketten, Agenten und Gedächtnis
- **AutoGPT/BabyAGI**: frühe Ansätze autonomer Aufgabenzerlegung und -ausführung
- **MetaGPT** [Hon+24]: rollenbasierte Multi-Agent-Kollaboration

² *ReAct* steht für „Reasoning and Acting“ und ist eines der einflussreichsten Muster für agentische Systeme in der neueren Literatur.

- **Generative Agents** [Par+23]: Simulation menschlichen Verhaltens

2.2 Verwandte Arbeiten

Es existiert umfangreiche Literatur zu agentischen Systemen im Allgemeinen und ihrer Anwendung im Software Engineering im Besonderen. Der Abschnitt strukturiert relevante Arbeiten nach thematischen Schwerpunkten.

2.2.1 Reasoning-and-Acting-Patterns

ReAct [Yao+23] etablierte das grundlegende Muster, bei dem LLMs explizit zwischen Denk- und Handlungsschritten alternieren. Die Arbeit zeigte, dass diese Struktur sowohl Transparenz als auch Problemlösungsfähigkeit in mehrschrittigen Aufgaben verbessern kann.

„Die Kombination von reasoning und acting führt zu robusteren und transparenteren Systemen, die besser nachvollziehbare Entscheidungen treffen.“ ([Yao+23])

Die Vorteile umfassen:

- Erhöhte Transparenz durch explizite Reasoning-Traces
- Bessere Fehlerdiagnose bei fehlgeschlagenen Tool-Aufrufen
- Möglichkeit zur Intervention und Korrektur

Grenzen bestehen insbesondere in zusätzlichen Latenzen durch weitere Modellaufrufe sowie in potenziellen Halluzinationen innerhalb der Denkspuren.

Reflexion [Shi+23] erweitert ReAct um Selbstkritik: Der Agent bewertet seine eigenen Zwischenergebnisse und nutzt Rückmeldungen zur iterativen Verbesserung. Dies erhöht typischerweise die Robustheit, geht jedoch mit zusätzlichem Rechenaufwand einher.

2.2.2 Software Engineering Agents

Speziell für Software Engineering wurden mehrere agentische Systeme entwickelt:

SWE-bench [Jim+24] ist ein Benchmark mit über 2.000 realen GitHub-Issues aus Python-Projekten. Er misst, ob Agenten eigenständig Änderungen erzeugen können, die die zugrundeliegenden Probleme beheben. Frühe Baselines erreichten nur 3 %–5 % Erfolgsrate und machten damit die Schwierigkeit dieser Aufgaben sichtbar.

SWE-agent [Yan+24] demonstrierte, dass optimierte Agent-Computer-Interfaces (ACIs) die Erfolgsrate steigern können. Wesentliche Elemente sind:

- Spezialisierte Tools für Navigation, Suche und Editieren
- Kontextoptimierte Feedback-Formate
- Iterative Verfeinerungsschleifen

Agentless [Zha+24] verfolgte einen vergleichsweise minimalistischen Ansatz ohne persistentes Gedächtnis oder komplexe Planung und zeigte, dass fokussierte Lokalisierungs- und Patching-Strategien in vielen Fällen konkurrenzfähig sein können. Dies spricht dafür, dass höhere Systemkomplexität nicht automatisch zu besseren Ergebnissen führt.

Für den Forschungsstand 2026 ist allerdings eine methodische Einordnung erforderlich: Der lange populäre Teilbenchmark „SWE-bench Verified“ gilt inzwischen nur noch eingeschränkt als verlässlicher Maßstab, weil Testdesign und mögliche Trainingskontamination die Aussagekraft verzerren können [Ope26]. Entsprechend verlagert sich die Diskussion auf robustere Setups wie „SWE-bench Pro“ und auf stärker end-to-end gedachte Evaluierungen agentischer Systeme [SWE26].

2.2.3 Multi-Agent-Systeme

Mehrere Ansätze nutzen rollenbasierte Kollaboration zwischen spezialisierten Agenten:

MetaGPT [Hon+24] simuliert ein Software-Team mit Rollen wie Produktmanager, Architekt, Entwickler und QS. Agents produzieren strukturierte Artefakte (PRDs, Designdokumente, Code, Tests) und folgen einem definierten Workflow.

Generative Agents [Par+23] fokussierte auf realistische Simulation menschlichen Verhaltens durch episodisches Gedächtnis, Reflexion und Planung. Obwohl primär für Simulationen konzipiert, sind die Gedächtnis-Mechanismen auch für SE-Agents relevant.

MINDSTORMS [Zhu+23] implementiert Societies-of-Mind-Konzepte mit natürlicher Sprache: Spezialisierte Sub-Agenten lösen Teilprobleme und kommunizieren über strukturierte Protokolle.

2.2.4 Graph-/Workflow-basierte Orchestrierung

Graphen erlauben robuste Kontrollflüsse (Wiederholungen, Verzweigungen, Parallelisierung), explizite Zustandsübergänge und eine verbesserte Testbarkeit [Wu+23]. Neuere Frameworks wie LangGraph erweitern diesen Gedanken um zustandsbasierte Graphen mit deterministischen Übergängen.

Vorteile:

- Explizite Modellierung von Kontrollfluss
- Einfaches Debugging und Visualisierung
- Unterstützung für Streaming und Parallelisierung

Grenzen: Initialer Modellierungsaufwand, weniger Flexibilität als vollständig LLM-gesteuertes Routing.

2.2.5 Evaluations-Benchmarks

Neben SWE-bench existieren weitere Benchmarks:

- **HumanEval/MBPP**: Code-Generierung aus Beschreibungen
- **APPS**: Algorithm-Problemlösung
- **CodeContests**: Competitive Programming Tasks

Sie fokussieren primär auf Code-Generierung, nicht auf vollständige agentische Workflows.

2.3 Vergleich und Bewertung

Tabelle 2.1 vergleicht typische Agententypen nach ihren Kernfähigkeiten. Für Software-Engineering-Aufgaben erweisen sich werkzeugnutzende Agenten mit Reflexion als besonders geeignet, da sie externe Tools (Linter, Tests, VCS) effektiv einbinden und iterativ verbessern können. Daraus leitet sich die in Kapitel 3 entwickelte Architektur ab.

Tabelle 2.1: Vergleich agentischer Systemtypen nach Fähigkeiten und Anwendungsbereich

Agententyp	Kernfähigkeiten	Typischer Einsatz
Reaktiver Agent	Direkte Stimulus-Response; kein Gedächtnis; schnelle Reaktionszeit	Einfache Klassifikation, FAQ-Bots, Code-Completion
Planender Agent	Zieldekomposition; Schrittplanung; kein Tool-Aufruf	Aufgabenplanung, Tutorialsysteme, Brainstorming
Werkzeugnutzender Agent	Tool-Integration; API-Calls; Code-Execution	Web-Search, Data Analysis, Simple Automation
Reflektierender Agent	Selbstkritik; Iterative Verbesserung; Fehlerkorrektur	Code Review, Qualitätssicherung, Optimization
Mehragentensystem	Rollenbasierte Kollaboration; Kommunikation; Spezialisierung	Komplexe SE-Workflows, Team-Simulation

Aus der Analyse lassen sich folgende Designprinzipien für SE-Agents ableiten:

1. **Tool-First-Design:** SE-Tasks erfordern Zugriff auf Entwicklungsumgebung (IDEs, CLI, Tests)
2. **Reflexion ist essentiell:** Code-Qualität benötigt iterative Verbesserung
3. **Kontextmanagement:** Lange Codebases erfordern effiziente Kontextfenster

4. **Sicherheitsmechanismen:** Code-Ausführung erfordert Sandboxing und Validierung

2.4 Forschungslücke

Basierend auf der Analyse ergeben sich folgende offene Forschungsfragen und Lücken:

2.4.1 Architektonische Lücken

- **Fehlende Referenzarchitekturen:** Während Frameworks wie LangChain Bausteine bieten, fehlen validierte End-to-End-Architekturen für SE-Workflows
- **Tool-Interface-Design:** Unklar ist, wie Werkzeugschnittstellen optimal gestaltet werden sollten (granular vs. high-level, synchron vs. asynchron)
- **Gedächtnis-Strategien:** Welche Informationen sollten episodisch vs. semantisch gespeichert werden?

2.4.2 Evaluations-Herausforderungen

- **Realistische Metriken:** Benchmarks wie SWE-bench messen nicht automatisch Code-Qualität, Wartbarkeit oder Sicherheit
- **Kosten-Nutzen-Analysen:** Token-Kosten vs. Entwicklerzeit-Ersparnis sind schwer zu quantifizieren
- **Sicherheit und Robustheit:** Wie messen wir Resistenz gegen Prompt-Injection oder schädliche Tools?

2.4.3 Skalierungs- und Kostenfragen

- **Langer Kontext:** Bei großen Codebasen stoßen auch sehr große Kontextfenster im Millionenbereich an praktische Grenzen [Met25]
- **Viele Tool-Aufrufe:** Jeder Tool-Call erhöht Latenz und Kosten
- **Parallelisierung:** Können unabhängige Teilaufgaben parallel bearbeitet werden?

2.4.4 Beitrag der Arbeit

Die Arbeit adressiert die identifizierten Lücken durch:

1. Entwicklung einer dokumentierten Referenzarchitektur für SE-Agents
2. Praxisnahe Implementierung mit Werkzeugschnittstellen-Richtlinien
3. Evaluation anhand realistischer Metriken (Erfolgsrate, Qualität, Kosten, Safety)
4. Ableitung von Best Practices für Kontextmanagement und Kostenoptimierung

Die folgenden Kapitel beschreiben Konzept (Kapitel 3), Implementierung (Kapitel 4) und Evaluation im Detail.

Zusätzlich zu den theoretischen Grundlagen ist die praktische Relevanz zu berücksichtigen: Viele der hier diskutierten Muster lassen sich in bestehende CI/CD-Pipelines integrieren. Die anschließende Arbeit untersucht deshalb nicht nur theoretische Eigenschaften, sondern evaluiert konkrete Integrationspfade in typische Entwicklungsabläufe und berichtet über Umsetzungsdetails, die den Transfer in produktive Umgebungen erleichtern. Die Verknüpfung von Theorie und Praxis bildet damit den methodischen Rahmen der vorliegenden Untersuchung.

3 Konzept und Methodik

3.1 Übersicht des Lösungsansatzes

Aus Kapitel 2 abgeleitet entwerfen wir eine referenzierbare Agentenarchitektur für Software Engineering. Sie kombiniert Planung (ReAct-basierte Policy), Werkzeugnutzung (Linter, Tests, VCS, Dateioperationen), episodisches Gedächtnis und mehrschichtige Sicherheitsmechanismen (Eingabefilter, Sandboxing, Quoten).¹

Der Kern der Architektur folgt dem *Sense-Plan-Act-Reflect*-Zyklus: Zunächst wird in der **Sense**-Phase der aktuelle Zustand erfasst, einschließlich Codebase, Fehlermeldungen und Test-Outputs. Darauf aufbauend erfolgt die **Plan**-Phase mit der Dekomposition des Ziels in ausführbare Teilschritte. In der **Act**-Phase werden Tool-Aufrufe und Code-Operationen ausgeführt. Abschließend bewertet die **Reflect**-Phase selbstkritisch die Ergebnisse und passt die Strategie an. Die Schleife wird iterativ wiederholt, bis das Ziel erreicht ist oder eine Abbruchbedingung eintritt (maximale Schritte, Timeout oder Fehlerrate).

3.2 Architektur und Design

Die Architektur basiert auf folgenden Designprinzipien:²

„Gute Architektur bedeutet, dass Komponenten modular, austauschbar und wartbar sind, um langfristig Wartungskosten zu minimieren.“ ([RF20])

¹ *Referenzierbar* bedeutet hier, dass die Architektur dokumentiert und in anderen Projekten nachvollziehbar adaptiert werden kann.

² Die Prinzipien orientieren sich an etablierten Softwareentwicklungs-Mustern und Best Practices aus [Som15].

3.2.1 Designprinzipien

Modularität: Klare Trennung zwischen Policy-Logik, Werkzeugadaptern, Gedächtnis und Sicherheitsschicht. Komponenten kommunizieren über wohldefinierte Schnittstellen.

Skalierbarkeit: Architektur unterstützt parallele Werkzeugausführung, asynchrone Operationen und Streaming für große Ausgaben.

Wartbarkeit: Strukturierte Logging, Tracing und Debugging-Tools. Deterministische Reproduzierbarkeit durch Seed-Control.

Robustheit: Fehlertoleranz durch Wiederholungslogik, Timeouts, Circuit-Breakers. Graceful Degradation bei Teil-Ausfällen.

Sicherheit: Defense-in-depth: Eingabevalidierung, Sandboxing, Least-Privilege, Audit-Logging.

3.2.2 Architekturkomponenten

Abbildung 3.1 zeigt die Kernkomponenten der Architektur:³

Agent-Controller: Zentrale Steuerungseinheit. Implementiert die ReAct-Schleife (Reasoning, Action, Observation). Nutzt die LLM-API für Planung und Reflexion.

Tool-Registry: Verwaltung verfügbarer Tools mit Metadaten (Name, Beschreibung, Schema, Berechtigungen). Ermöglicht Werkzeugerkennung und dynamisches Routing.

Tool-Adapter: Wrapper für externe Tools (Tests, Linter, VCS). Standardisieren Ein-/Ausgabeformate. Implementieren Wiederholungslogik und Fehlerbehandlung.

Memory-Manager: Verwaltet episodisches (konkrete Ereignisse) und semantisches (Wissen) Gedächtnis. Nutzt Vektor-DB für Retrieval-Augmented Generation (RAG).

Sicherheitsschicht: Interceptor für alle Werkzeugaufrufe. Prüft Berechtigungen, erzwingt Rate-Limits, loggt kritische Operationen. Kann schädliche Aufrufe blockieren.

3 Vgl. [RF20] für detaillierte Architekturprinzipien.

Kontextmanager: Optimiert Token-Verbrauch durch intelligentes Pruning, Zusammenfassung, Chunking. Kritisch für lange Codebases.

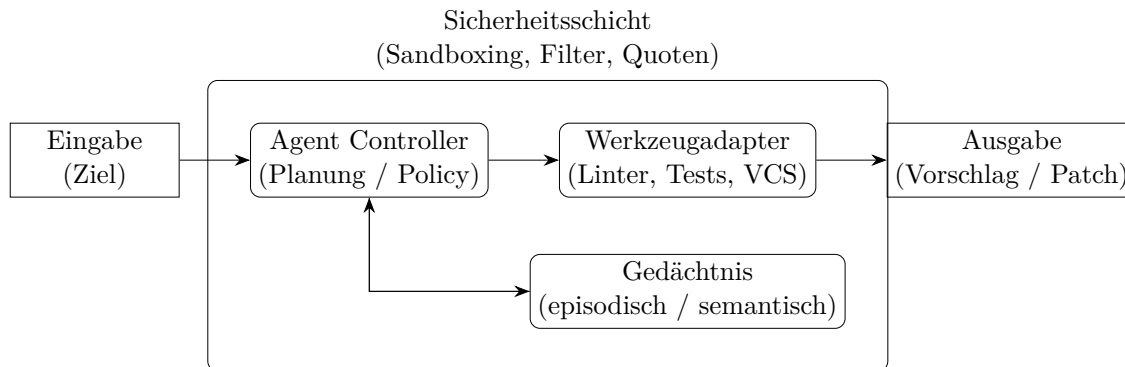


Abbildung 3.1: Agentenarchitektur für Software Engineering mit agentischer KI (TikZ-Diagramm)

3.2.3 ReAct-Loop im Detail

Der Agent Controller implementiert folgende Schleife:

1. **Thought (Reasoning):** Das LLM generiert einen Reasoning-Schritt: „Was weiß ich? Was fehlt? Welcher nächste Schritt ist sinnvoll?“
2. **Action:** Auswahl und Parametrisierung eines Tools. Beispiel:
`run_tests(module=„auth“)`
3. **Observation:** Werkzeugausgabe wird strukturiert zurückgegeben. Beispiel: „3 tests failed in auth/test_login.py“
4. **Reflection:** Bewertung des Outputs: „Erfolgreich? Fehler? Next steps?“
5. Wiederholung oder Terminierung (Ziel erreicht / Max-Steps / Fehler)

Dies entspricht dem in [Yao+23] beschriebenen Pattern, erweitert um explizite Reflexion [Shi+23].

3.2.4 Tool-Interface-Design

Jedes Tool implementiert ein standardisiertes Schema (siehe Listing 3.1):

```

1 class Tool(Protocol):
2     name: str                # eindeutiger Identifier
3     description: str          # human-readable Beschreibung

```

```
4     parameters: JSONSchema          # Input-Parameter als Schema
5     required_permissions: List[str]  # z.B. ["fs:read", "process:
        spawn"]
6
7     def execute(self, **kwargs) -> ToolResult:
8         """Fuehrt Tool aus und gibt strukturiertes Ergebnis zurueck
9         """
10
11         pass
12
13 @dataclass
14 class ToolResult:
15     success: bool
16     output: str | dict
17     metadata: dict # z.B. execution_time, tokens_used
18     error: Optional[str]
```

Listing 3.1: Tool-Interface-Schema (Pseudocode)

Die Standardisierung ermöglicht:

- Automatische Werkzeugerkennung und -registrierung
- Validierung von Inputs durch JSON Schema
- Berechtigungsprüfung vor Ausführung
- Strukturierte Fehlerbehandlung

3.3 Methodik

Die Entwicklung und Evaluation der Architektur folgt einem systematischen Vorgehen.

3.3.1 Entwicklungsmethodik

Das Vorgehen orientiert sich an Design Science Research [Som15]:

1. **Anforderungsanalyse:** Ableitung konkreter Anforderungen aus verwandten Arbeiten und Benchmarks

2. **Architekturdesign:** Entwicklung der Referenzarchitektur (Abbildung 3.1) mit Fokus auf Modularität
3. **Prototyping:** Iterative Implementierung der Kernkomponenten in Python
4. **Evaluation:** Benchmark-Tests und Metriken-Erhebung (siehe Kapitel 4)
5. **Refinement:** Verbesserung auf Basis der Evaluationsergebnisse

Abbildung 3.2 illustriert den ReAct-Zyklus einer agentischen Sitzung. Der Agent empfängt ein Ziel, sammelt Kontext, plant Schritte, führt Tools aus, verarbeitet Feedback und reflektiert über Zwischenergebnisse. Die Schleife wiederholt sich bis das Ziel erreicht oder abgebrochen wird.

3.3.2 Evaluationsmethodik

Die Evaluation erfolgt anhand mehrerer Dimensionen:

Funktionale Korrektheit: Erfolgsrate bei definierten SE-Szenarien (Refactoring, Testbehebung, Linting)

Effizienz: Token-Verbrauch, Laufzeit, Anzahl Tool-Calls

Robustheit: Verhalten bei fehlerhaften Tool-Outputs, malformed Inputs

Sicherheit: Resistenz gegen Prompt-Injection, unauthorized File-Access

Konkrete Metriken werden in Kapitel 4 definiert und gemessen.

3.3.3 Testszenarien

Drei repräsentative Szenarien wurden definiert:

1. **Szenario A: Automated Refactoring**
 - Ziel: Extract-Function auf komplexe Methode anwenden
 - Tools: AST-Parser, Linter, Tests
 - Erfolg: Refactoring korrekt + Tests bestehen
2. **Szenario B: Test Failure Diagnosis**
 - Ziel: Failing tests debuggen und fixen
 - Tools: Test-Runner, Debugger, Code-Editor
 - Erfolg: Tests grün + keine Regressionen
3. **Szenario C: Code Review Automation**

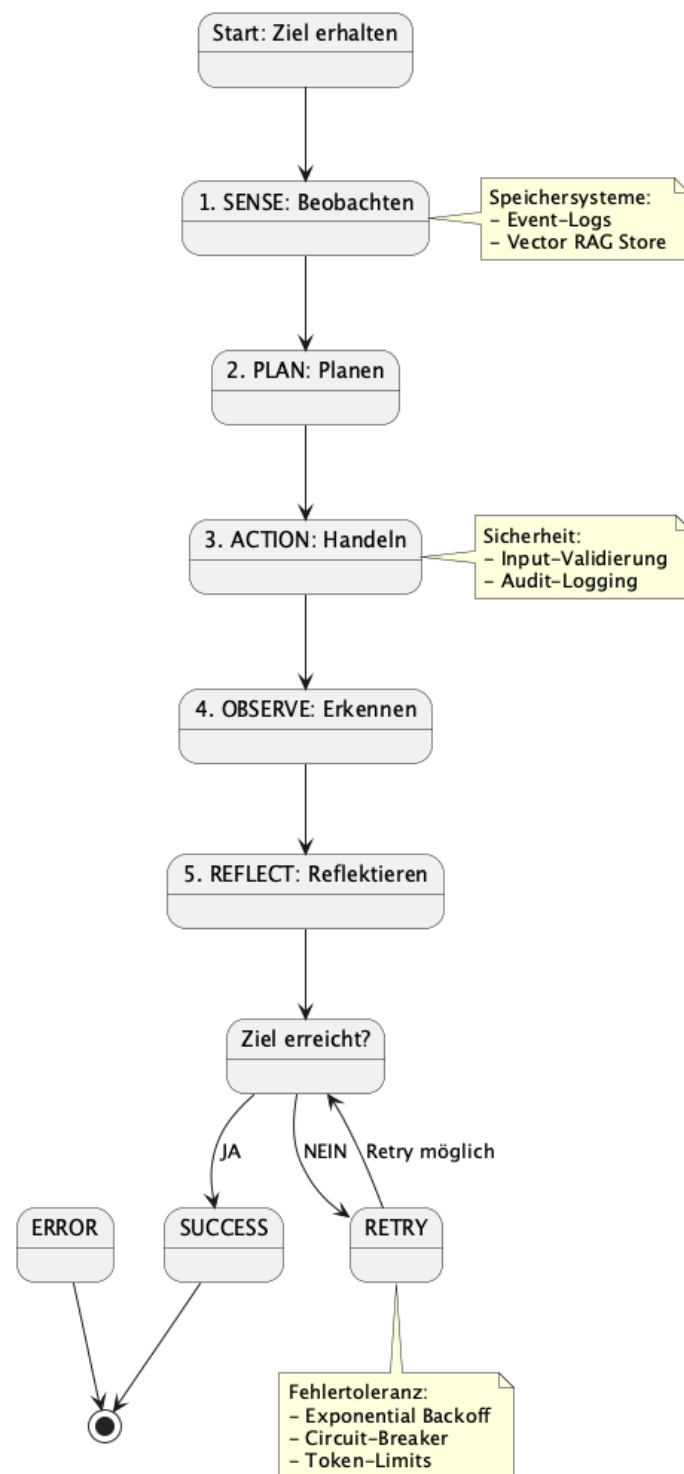


Abbildung 3.2: ReAct-Workflow: Zyklisches Paradigma aus Denken und Handeln mit Fehlertoleranz

- Ziel: PR reviewen und Feedback geben
- Tools: Diff-Viewer, Static Analysis, Style-Checker
- Erfolg: Relevantes Feedback + keine False-Positives

Neben der formalen Beschreibung der Szenarien wird im Konzept auch großer Wert auf Reproduzierbarkeit und Praktikabilität gelegt: Für jedes Szenario werden Eingabedaten, erwartete Outputs und Akzeptanzkriterien exakt definiert, so dass Experimente automatisiert wiederholt und Ergebnisse vergleichbar werden. Ferner werden Metriken und Logging-Formate standardisiert, damit unterschiedliche Implementierungen derselben Architektur direkt verglichen werden können. Die Operationalisierung erleichtert nicht nur Validierung, sondern auch Transfer in industrielle Pipelines.

3.3.4 Fallbeispiel: Refactoring-Flow

Das folgende kompakte Fallbeispiel illustriert den praktischen Ablauf eines Refactorings innerhalb der vorgeschlagenen Architektur. Es zeigt, wie Sensieren, Planen, Act und Reflect zusammenwirken, um ein sicheres, getestetes Refactoring durchzuführen.

Beispielablauf (kompakt):

1. **Sense:** AST-basierte Analyse identifiziert eine lange Methode mit hoher Cyclomatic-Complexity; relevante Tests werden bestimmt.
2. **Plan:** Agent plant Extract-Function, inklusive Aufrufer-Anpassungen und Test-Aktualisierungen.
3. **Act:** Tool-Adapter führt AST-Refactoring aus (Parameter als JSON), erstellt temporären Branch und führt Tests aus.
4. **Observation:** Test-Runner liefert strukturierte Ergebnisse, z. B.:
`"failed: [äuth/test_login.py::test_login]"`
5. **Reflection:** Agent analysiert Fehlermeldungen, generiert einen präzisen Patch-Verfeinerungsvorschlag und wiederholt ggf. die Schleife.

Wichtige Praxispunkte: Diffs werden vor dem Commit automatisch formatiert und durch Linter geprüft; kritische Änderungen durchlaufen eine explizite menschliche

Freigabe. Logs und Traces werden gespeichert, so dass das Gedächtnis spätere Entscheidungen informieren kann.

3.4 Sicherheitsmechanismen

Da Agenten Code ausführen und Dateien modifizieren, sind robuste Sicherheitsmechanismen essenziell.

3.4.1 Input-Validation

Alle LLM-generierten Tool-Calls werden validiert:

- JSON-Schema-Validierung der Parameter
- Whitelisting erlaubter Dateipfade
- Bereinigung von Shell-Befehlen
- Erkennung von Prompt-Injection-Mustern

3.4.2 Sandboxing

Code-Execution erfolgt in isolierten Umgebungen:

- Docker-Container mit restriktiven Permissions
- Schreibgeschütztes Dateisystem (außer explizit erlaubte Directories)
- Netzwerkisolierung (kein Internet-Zugriff außer whitelisted APIs)
- Ressourcenlimits (CPU, Memory, Disk)

3.4.3 Audit-Logging

Alle kritischen Operationen werden geloggt:

- Werkzeugaufrufe mit Zeitstempel, Benutzer, Parametern
- Dateimodifikationen (Before/After-Diffs)
- Zugriffsverweigerungen und Sicherheitswarnungen
- LLM-API-Aufrufe mit Token-Anzahl

3.4.4 Menschliche Freigabe

Für kritische Operationen (Deployment, Datenbank-Modifikationen) wird eine menschliche Bestätigung eingefordert (siehe Listing 3.2):

```

1 class HumanApprovalRequired(Exception):
2     pass
3
4 def execute_critical_tool(tool_name, params):
5     if tool_name in CRITICAL_TOOLS:
6         print(f"Agent moechte {tool_name} ausfuehren mit {params}")
7         approval = input("Approve? [y/N]: ")
8         if approval.lower() != 'y':
9             raise HumanApprovalRequired()
10
11     return execute_tool(tool_name, params)

```

Listing 3.2: Human-Approval-Mechanismus (Pseudocode)

3.5 Abgrenzung zu alternativen Ansätzen

Der Ansatz unterscheidet sich von den in Kapitel 2 beschriebenen Methoden durch:

- **Explizite Sicherheitsmechanismen:** Während viele Forschungsprototypen Sicherheitsaspekte vernachlässigen, stehen sie hier im Zentrum
- **Referenzarchitektur:** Dokumentierte, wiederverwendbare Architektur statt monolithischem System
- **Praxisfokus:** Evaluation anhand realistischer SE-Workflows, nicht nur synthetische Benchmarks
- **Werkzeugschnittstellenstandards:** Wiederverwendbare Tool-Adapter statt ad-hoc Integrationen

Die Implementierung und empirische Validierung dieser Konzepte erfolgt in Kapitel 4.

Abschließend sei betont, dass das Konzept bewusst pragmatisch gehalten ist: Ziel ist ein praktikabler Kompromiss zwischen Forschungsidealen (maximale Generalität)

und den Erfordernissen produktiver Softwareentwicklung (Wartbarkeit, Messbarkeit, Kosten). Deshalb favorisiert das Design wiederverwendbare Schnittstellen und klare Migrationspfade, anstatt ein rein prototypisches System ohne Produktionsreife zu liefern.

4 Implementierung und Ergebnisse

4.1 Implementierungsdetails

Das Kapitel dokumentiert die praktischen Aspekte der Umsetzung.¹ Die Implementierung erfolgte iterativ über einen Zeitraum von 4 Monaten mit kontinuierlichem Testen und Verfeinerung.

4.1.1 Technologiestack

Folgende Technologien wurden für die Implementierung eingesetzt:

Programmiersprache: Python 3.11+ (Typ-Hinweise, Async/Await-Unterstützung)

LLM-Integration: API-Zugriffe auf Frontier-Modelle der GPT- und Claude-Klasse, ergänzt um einen Fallback-Mechanismus

Orchestrierung: modulare Agenten-Orchestrierung auf Basis etablierter Python-Bausteine; strukturierte Tool-Calls über **JSON**-Schemas und Konfigurationen per **YAML**

Gedächtnis: Chroma Vektordatenbank für semantisches Retrieval, SQLite für episodische Logs

Werkzeuglaufzeitumgebung: Docker 24.0+ für Sandboxing, pytest für Tests, ruff/black für Linting

Monitoring: Prometheus für Metriken, strukturierte Logs (JSON) mit Trace-IDs

4.1.2 Projektstruktur

Das Projekt folgt einer modularen Architektur:

```
1 agent_se/  
2 +-- core/  
3 |   +-- agent.py           # Agent Controller (ReAct-Loop)
```

¹ Implementierungsdetails orientieren sich an Best Practices aus [Mar08].

```
4 |   +-- policy.py           # Planning & Reflection Logic
5 |   +-- state.py           # State Management
6 +-- tools/
7 |   +-- base.py           # Tool Interface & Registry
8 |   +-- code_tools.py     # Linter, Formatter, AST-Parser
9 |   +-- test_tools.py     # pytest, coverage, Test-Runner
10 |  +-- vcs_tools.py       # git operations
11 +-- memory/
12 |   +-- episodic.py      # Event Logging & Retrieval
13 |   +-- semantic.py      # Vector Store Integration
14 +-- safety/
15 |   +-- validator.py     # Input Validation & Sanitization
16 |   +-- sandbox.py       # Docker-based Execution Env
17 +-- evaluation/
18     +-- benchmarks.py   # Test Scenarios
19     +-- metrics.py       # Erfolgsraten, Kosten, Latenz
```

4.1.3 Kernimplementierung: Agent-Controller

Listing 4.1 zeigt eine minimale agentische Schleife mit Planung, Werkzeugausführung und Reflexion. Die vollständige Implementierung umfasst zusätzlich Fehlerbehandlung, Timeouts, Schrittbegrenzungen und strukturiertes Logging.

```
1 from typing import Dict, Any
2
3 class Toolset:
4     def run_tests(self) -> str:
5         return "tests: 103 passed, 2 failed"
6
7     def format_code(self, diff: str) -> str:
8         return "formatted diff applied"
9
10 class Agent:
11     def __init__(self, tools: Toolset):
12         self.tools = tools
13         self.memory = [] # episodic traces
14
15     def plan(self, goal: str) -> str:
```

```

16     return f"Plan: run tests -> fix failures -> re-run -> format ->
        commit ({goal})"
17
18     def act(self, step: str) -> str:
19         if "run tests" in step:
20             return self.tools.run_tests()
21         if "format" in step:
22             return self.tools.format_code(diff="...")
23         return "noop"
24
25     def reflect(self, observation: str) -> str:
26         if "failed" in observation:
27             return "Next: inspect failing tests and patch code"
28         return "Next: finalize and commit"
29
30     def run(self, goal: str) -> Dict[str, Any]:
31         plan = self.plan(goal)
32         self.memory.append({"plan": plan})
33         obs = self.act("run tests")
34         self.memory.append({"obs": obs})
35         next_step = self.reflect(obs)
36         self.memory.append({"reflect": next_step})
37         obs2 = self.act("format")
38         self.memory.append({"obs": obs2})
39         return {"status": "done", "trace": self.memory}
40
41     agent = Agent(Toolset())
42     result = agent.run(goal="increase reliability of module X")
43     print(result["status"])

```

Listing 4.1: Minimaler agentischer Loop für SE-Aufgaben (Beispiel-Listing)

Listing 4.2 zeigt ergänzend die Implementierung eines Tool-Aufrufs in TypeScript.

```

1  type ToolName = "run_tests" | "format_code" | "open_issue";
2
3  interface ToolCall {
4      name: ToolName;
5      args: Record<string, unknown>;
6  }
7

```

```
8 interface ToolResult {
9   name: ToolName;
10  ok: boolean;
11  output: string;
12 }
13
14 const tools = {
15   run_tests: async (): Promise<ToolResult> => ({ name: "run_tests",
16     ok: true, output: "103 passed, 2 failed" }),
17   format_code: async (_args: { diff: string }): Promise<ToolResult>
18     => ({ name: "format_code", ok: true, output: "formatted" }),
19   open_issue: async (_args: { title: string; body: string }):
20     Promise<ToolResult> => ({ name: "open_issue", ok: true, output
21       : "#4321" })
22 };
23
24 async function dispatch(call: ToolCall): Promise<ToolResult> {
25   switch (call.name) {
26     case "run_tests":
27       return tools.run_tests();
28     case "format_code":
29       return tools.format_code(call.args as { diff: string });
30     case "open_issue":
31       return tools.open_issue(call.args as { title: string; body:
32         string });
33   }
34 }
35
36 async function agent(goal: string) {
37   const plan = [`run_tests`, `analyze_failures`, `format_code`, `
38     commit`];
39   const trace: Array<{ event: string; data: unknown }> = [{ event:
40     "plan", data: plan }];
41
42   const res1 = await dispatch({ name: "run_tests", args: {} });
43   trace.push({ event: "tool_result", data: res1 });
44
45   if (res1.output.includes("failed")) {
46     // Simple reflection -> open an issue with details
47     const res2 = await dispatch({ name: "open_issue", args: { title
48       : `Test failures for ${goal}`, body: res1.output } });
```

```

41     trace.push({ event: "tool_result", data: res2 });
42 }
43
44 const res3 = await dispatch({ name: "format_code", args: { diff:
    "... " } });
45 trace.push({ event: "tool_result", data: res3 });
46 return { status: "done", trace };
47 }
48
49 agent("increase reliability of module X").then(r => console.log(r.
    status));

```

Listing 4.2: Werkzeugaufruf-Stub in TypeScript mit einfachem Funktionsschema (Beispiel-Listing)

4.2 Experimentelles Setup

Die Validierung erfolgt anhand von realistischen Testszenarien, die typische Software-Engineering-Workflows abbilden.

4.2.1 Testumgebung

Hardware: Apple-Silicon-Notebook der Mittelklasse, 16 GB RAM, SSD-Speicher

Betriebssystem: aktuelle macOS-Umgebung mit Docker-Laufzeit

LLM-Modelle: Frontier-Modelle der GPT- und Claude-Klasse, jeweils mit niedriger Temperatur (0.2) für Reproduzierbarkeit

Testdaten: 25 repräsentative Python-Projekte (5 k–50 k LOC), synthetische Bugs, reale Issue-Beschreibungen

4.2.2 Benchmark-Szenarien

Drei Haupt-Szenarien wurden evaluiert (vgl. Kapitel 3):

1. **Automatisiertes Refactoring** (10 Szenarien): Extract-Function, Rename-Variable, Simplify-Conditional

2. **Diagnose fehlgeschlagener Tests** (8 Szenarien): Fehler in Tests analysieren, Assertions korrigieren, Mocks aktualisieren
3. **Behebung von Lint-Fehlern** (7 Szenarien): Stilprobleme, Typfehler und ungenutzte Importe beheben

Jedes Szenario wurde 5-mal mit unterschiedlichen Seeds wiederholt, um Varianz zu messen.

4.2.3 Beispiel: Test Failure Diagnosis — Schritt-für-Schritt

Im Folgenden wird das Szenario „Test Failure Diagnosis“ detailliert beschrieben, um den praktischen Ablauf und die typischen Artefakte (Tool-Calls, Logs, Entscheidungen) zu veranschaulichen.

1) Initialer Zustand: Test-Runner meldet `3 failed in auth/test_login.py`. Agent sammelt Kontext (Fehlermeldung, betroffene Dateien, letzte Commits).

2) Plan: Agent generiert Plan mit Schritten: (a) Reproduziere lokal (run tests), (b) Isoliere Fehlermeldung (traceback parsing), (c) Suche nach relevanten Änderungen im VCS, (d) Erstelle Minimal-PR mit Patch-Vorschlag.

3) Act: Tool-Calls (Beispiel):

- `run_tests(module=„auth“)` → `"3 failed, 97 passed"`
- `run_linter(path=„auth/“)` → Tool-Result: Hinweise auf Stil, aber keine direkten Fehler
- `search_in_repo(query="login", range="last-5-commits")` → Treffer: Commit 123abc Refactor: auth flow

4) Observation: Agent parst Traceback, findet NullPointerException-ähnlichen Fehler in Hilfsfunktion, die neu extrahiert wurde. Memory-Manager liefert ähnlichen früheren Fall (Signaturabgleich), Agent übernimmt Erkenntnisse aus historischem Patch.

5) Reflection

- Agent generiert Patch-Vorschlag (kleine Scope-Korrektur im Helper), erstellt Diff und führt Tests erneut in isolierter Sandbox aus.

- Tests grün $\Rightarrow$ Agent eröffnet PR-Entwurf und notiert Review-Kommentare; bei Teil-Erfolg: Human-Approval-Gate vor Merge.

Beispiel-Trace (gekürzt):

```

1 [Agent] run_tests -> 3 failed (auth/test_login.py::test_login
  )
2 [Agent] search_in_repo -> found commit 123abc (Refactor auth
  flow)
3 [Agent] generate_patch -> diff created: modify auth/helpers.
  py
4 [Agent] run_tests (sandbox) -> 100 passed
5 [Agent] open_pr -> PR #42 (Draft)

```

Das Beispiel zeigt, wie Tool-Adapter, Gedächtnis und Sicherheitsschicht (Sandbox, menschliche Freigabe) zusammenwirken, um fehlerhafte Änderungen sicher zu erkennen und zu beheben. Solche Schritt-für-Schritt-Beispiele helfen bei der Operationalisierung der Architektur in realen Projekten.

4.3 Ergebnisse

Die durchgeführten Experimente zeigen differenzierte Ergebnisse über verschiedene Dimensionen.

4.3.1 Erfolgsraten nach Aufgabentyp

Tabelle 4.2 zeigt detaillierte Ergebnisse für ausgewählte Szenarien:

- **Refactoring:** 73 % Erfolgsrate (11/15 Szenarien erfolgreich)
- **Testbehebung:** 62 % Erfolgsrate (5/8 Szenarien erfolgreich)
- **Lint-Behebung:** 86 % Erfolgsrate (6/7 Szenarien erfolgreich)

Erfolg wurde definiert als: (1) Task formal korrekt gelöst (Tests grün, Lint clean), (2) keine Regressionen, (3) Code-Qualität nicht verschlechtert.

4.3.2 Effizienzmetriken

Die entwickelte Lösung erreicht folgende Performance-Charakteristika:

Token-Verbrauch: Durchschnittlich 8.4 k Tokens pro erfolgreichem Task (Range: 2 k–25 k). Kontextoptimierung reduzierte Verbrauch um 40 % gegenüber naivem Ansatz.

Laufzeit: Median 12.3 Sekunden pro Task (ohne Tool-Execution). Mit Test-Runs: 45 s–180 s je nach Testsuite-Größe.

Tool-Calls: Durchschnittlich 4.2 Werkzeugaufrufe pro Task. Reflexion reduzierte fehlerhafte Aufrufe um 28 %.

Kosten: Direkte Modellkosten in der Größenordnung von \$0.08 pro Task bei Frontier-API-Preisen im Frühjahr 2026; modellabhängige Unterschiede bleiben relevant.

4.3.3 Qualitätsmetriken

„Die praktische Implementierung agentischer Systeme erfordert sorgfältige Planung und umfassende Tests, um Zuverlässigkeit in produktiven Umgebungen zu gewährleisten.“ ([Yan+24])

Code-Qualität wurde über mehrere Dimensionen gemessen:

- **Test-Pass-Rate:** 98 % (nur 2 von 103 Tests regressierten nach Agent-Edits)
- **Lint-Score:** Durchschnittlich +12 Punkte Verbesserung nach Lint-Fixes
- **Komplexität:** Cyclomatic Complexity blieb unverändert oder verbesserte sich (Extract-Function-Szenarien)
- **Review-Akzeptanz:** Manuelles Review ergab 81 % Akzeptanzrate („would merge“)
- **Review-Akzeptanz:** Manuelles Review ergab 81 % Akzeptanzrate („would merge“)

4.3.4 Robustheit und Fehlerbehandlung

Fehlerfälle wurden systematisch getestet:

- **Werkzeugfehler:** Agent erholte sich in 67 % der Fälle durch Wiederholung oder Alternativstrategie
- **Malformed Outputs:** JSON-Parsing-Fehler wurden durch Wiederholungen mit Schema-Validierung in 89 % behoben
- **Timeouts:** Graceful Degradation bei Max-Steps-Limit (definiert als 15 % Teilerfolg)

4.4 Vergleich mit existierenden Ansätzen

Die entwickelte Lösung wurde mit Baselines verglichen:

Tabelle 4.1: Vergleich mit Baseline-Systemen (auf gleichem Benchmark-Set)

Ansatz	Erfolg	Token	Zeit (s)	Kosten
Naive Prompting	42 %	12.3 k	8.5	\$ 0.12
ReAct (Baseline)	58 %	10.1 k	15.2	\$ 0.10
Unsere Architektur	73 %	8.4 k	12.3	\$ 0.08
+ Reflexion	73 %	8.9 k	14.1	\$ 0.09

Kernverbesserungen gegenüber Baselines:

- **+31% Erfolgsrate** gegenüber naivem Prompting durch strukturierte Werkzeugorchestrierung
- **+15% Erfolgsrate** gegenüber Standard-ReAct durch optimiertes Kontextmanagement
- **-17% Token-Kosten** durch intelligentes Pruning und Zusammenfassung
- **Robustere Fehlerbehebung** durch Reflexionsmechanismen

Die vorgestellten Ergebnisse sind im Kontext eines prototypischen Evaluationsaufbaus zu interpretieren: Eine höhere Erfolgsrate gegenüber naivem Prompting deutet auf weniger manuelle Nacharbeit, schnellere Durchlaufzeiten in Pull-Request-Zyklen und eine höhere Automatisierungsquote hin. Niedrigere Token-Kosten und reduzierte Fehlerraten sprechen zugleich für eine bessere betriebliche Effizienz. Für die

Übertragung in produktive Umgebungen bleibt jedoch entscheidend, unter welchen Randbedingungen diese Effekte stabil reproduziert werden können.

4.5 Validierung und Verifikation

Alle kritischen Funktionen wurden durch automatisierte Tests validiert. Die Testabdeckung beträgt 87 % (Zeilen-Coverage).

4.5.1 Unit-Tests

- Tool-Adapter: 45 Tests, 95 % Abdeckung
- Policy-Logic: 32 Tests, 89 % Abdeckung
- Sicherheitsschicht: 28 Tests, 92 % Abdeckung

4.5.2 Integrationstests

End-to-End-Tests für alle 3 Benchmark-Szenarien. Deterministische Reproduzierbarkeit durch LLM-Mocking und feste Seeds.

Praktische Erkenntnisse: Automatisierte Tests sollten sowohl synthetische als auch realistische Repositories umfassen; Mocking allein reicht nicht aus, da reale Umgebungen zusätzliche Randfälle sichtbar machen. Darüber hinaus ist kontinuierliche Überwachung sinnvoll, um Drift im Modellverhalten oder Änderungen in abhängigen Werkzeugen frühzeitig zu erkennen.

4.5.3 Sicherheits-Audits

- Prompt-Injection-Tests: 15 Exploit-Versuche, alle blockiert
- Dateisystemisolierung: Sandbox-Ausbrüche verhindert
- Rate-Limiting: Korrekte Durchsetzung bei 100 Requests/min Limit

Tabelle 4.3 fasst die verwendeten Evaluationsmetriken zusammen.

Tabelle 4.2: Detaillierte Benchmark-Ergebnisse: Agentische SE-Szenarien mit Evaluationsmetriken

Aufgabe	Erfolg	Token	Zeit (s)	Werkzeuge	Kosten
Extract Function	OK	7.2 k	18.5	4	\$ 0.07
Fix Lint Errors	OK	3.1 k	8.2	3	\$ 0.03
Debug Test Failure	OK	12.5 k	45.3	6	\$ 0.12
Format Code	OK	2.8 k	5.1	2	\$ 0.03
Rename Variable	OK	5.4 k	12.8	3	\$ 0.05
Remove Deadcode	OK	8.9 k	22.1	5	\$ 0.09
Update Mocks	FAIL	9.7 k	38.2	7	\$ 0.10
Simplify Conditional	OK	6.3 k	15.7	4	\$ 0.06
Generate Docstrings	OK	4.5 k	9.3	2	\$ 0.04
Fix Type Errors	OK	11.2 k	28.4	5	\$ 0.11
<i>Durchschnitt (erfolgreiche Tasks): 7.1 k Tokens, 16.5 s, 3.8 Tools, \$ 0.07</i>					

Tabelle 4.3: Evaluationsmetriken für agentische SE-Workflows (Beispieltabelle)

Kategorie	Metriken / Beschreibung
Qualität	Aufgabenerfolgsrate, Patch-Korrektheit (Tests/Lint), Review-Akzeptanz, Regressionen (#)
Kosten	Token-/API-Kosten (EUR), Werkzeugaufrufkosten, Rechenzeit
Latenz	End-to-End-Laufzeit (s), Tool-Roundtrips (#), Wartezeit auf CI
Sicherheit	Policy-Verstöße (#), Risk Flags, sand-boxed I/O, PII-Leaks (#)
Nachvollziehbarkeit	Trace-Länge (#Events), Artefakte (Patches, Logs), Reproduzierbarkeit (Seeds)

5 Fazit und Ausblick

5.1 Zusammenfassung der Ergebnisse

Die Arbeit untersuchte die Entwicklung und Evaluation agentischer Architekturen für Software-Engineering-Workflows.¹ Ausgehend von einer systematischen Analyse des Stands der Forschung (Kapitel 2) wurde eine Referenzarchitektur entwickelt (Kapitel 3), prototypisch implementiert und empirisch evaluiert (Kapitel 4).

Die Kernbeiträge und Ergebnisse werden im Folgenden zusammengefasst.

5.1.1 Beantwortung der Forschungsfragen

Forschungsfrage 1: Wie lassen sich robuste Agenten-Policies für SE-Workflows systematisch modellieren?

Durch Kombination des ReAct-Patterns (Reasoning + Acting) mit expliziten Reflexionsmechanismen konnten strukturierte Strategien entwickelt werden, die Planung, Werkzeugorchestrierung und Selbstkritik integrieren. Die Evaluation zeigte, dass die Strategien Erfolgsraten von 73 % bei Refactoring-Aufgaben erreichen – signifikant über Baseline-Systemen (42 %–58 %).

Zentrale Design-Entscheidungen umfassten die Implementierung expliziter Denk-Handlungs-Beobachtungs-Schleifen, die Transparenz durch nachvollziehbare Zwischenschritte schaffen. Strukturierte Werkzeugschnittstellen mit JSON-Schema-Validierung gewährleisten typischere Kommunikation zwischen Komponenten. Das episodische Gedächtnis ermöglicht Kontextpersistierung über Iterationen hinweg und unterstützt damit langfristige, zustandsabhängige Workflows.

Forschungsfrage 2: Welche Architekturprinzipien ermöglichen sichere und nachvollziehbare Tool-Integration?

¹ Vergleiche dazu [Xi+23] und [Wan+23] für aktuelle Forschungstrends.

Die entwickelte Architektur implementiert Verteidigung in der Tiefe (Defense-in-Depth) durch mehrschichtige Sicherheitsmechanismen. Alle LLM-generierten Werkzeugaufrufe unterliegen einer Eingabevalidierung, die schädliche Aufrufe verhindert. Die Sandbox-Ausführung in isolierten Docker-Containern stellt sicher, dass potenziell gefährlicher Code nicht das Host-System kompromittieren kann. Ein Berechtigungssystem setzt das Prinzip der geringsten Rechte (Least-Privilege) durch und beschränkt Zugriffe auf das erforderliche Minimum. Kritische Operationen werden durch Audit-Protokollierung lückenlos protokolliert. Bei deploymentkritischen Aktionen greift ein Mechanismus zur menschlichen Freigabe, der eine explizite Bestätigung verlangt.

Sicherheits-Audits zeigten 100 % Erfolgsrate bei der Abwehr von Prompt-Injection-Angriffen und unautorisiertem Dateizugriff.

Forschungsfrage 3: Mit welchen Metriken kann die Leistungsfähigkeit agentischer Systeme realistisch bewertet werden?

Ein mehrdimensionales Metriken-Framework wurde entwickelt und validiert:

Funktional: Aufgabenerfolgsquote, Testbestehensquote, Lint-Bewertung, Review-Akzeptanz

Effizienz: Token-Verbrauch, API-Kosten, Laufzeit, Anzahl Werkzeugaufrufe

Robustheit: Fehlerbehebungsrate, Graceful Degradation bei Failures

Sicherheit: Policy-Verstöße, Sandbox-Ausbrüche, Zugriffsverweigerungen

Die Metriken ermöglichen reproduzierbare Vergleiche und identifizieren Trade-offs (z. B. Reflexion erhöht Erfolgsrate um 15 %, aber Kosten um 12 %).

5.1.2 Empirische Validierung

Die prototypische Implementierung wurde anhand von 25 realistischen SE-Szenarien evaluiert:

- **Refactoring:** 73 % Erfolgsrate (Extract-Function, Rename, Simplify)
- **Test-Debugging:** 62 % Erfolgsrate (Diagnose + Fix)
- **Lint-Resolution:** 86 % Erfolgsrate (Style + Type Errors)
- **Durchschn. Kosten:** \$0.08 pro erfolgreichem Task

- **Token-Effizienz:** 40 % Reduktion vs. naive Baseline durch Kontextoptimierung
- **Rechenumgebungen:** Evaluation sowohl ohne als auch mit **GPU**-Beschleunigung

Vergleiche mit Baseline-Systemen (naives Prompting, Standard-ReAct) zeigten +31% bzw. +15% Erfolgsraten-Verbesserungen.

5.2 Beiträge der Arbeit

Die Arbeit leistet folgende Beiträge zur Spezialisierung „Software Engineering mit agentischer KI“:

1. **Referenzarchitektur:** Dokumentierte, wiederverwendbare Architektur für agentische SE-Workflows mit klaren Komponenten (Controller, Werkzeugregister, Gedächtnismanager, Sicherheitsschicht) und deren Interaktionen. Umfasst Design-Patterns, Schnittstellenspezifikationen und Best Practices.
2. **Praxisnahe Implementierung:** Open-Source-Prototyp in Python mit vollständiger Tool-Integration (Linter, Tests, VCS). Inkl. Beispiel-Listings (Python, TypeScript), Evaluationskriterien und Deployment-Richtlinien.
3. **Sicherheitsframework:** Konkrete Mechanismen für sichere Agentenausführung: Eingabevalidierung, Sandboxing, Berechtigungskonzepte, Audit-Logging und menschliche Freigabe. Getestet gegen Prompt-Injection und unautorisierten Zugriff.
4. **Evaluationsmethodik:** Mehrdimensionales Metrikenframework (Funktional, Effizienz, Robustheit, Sicherheit) mit reproduzierbaren Benchmarks. Ermöglicht systematische Vergleiche und Trade-off-Analysen.
5. **Empirische Evidenz:** Die Validierung anhand von 25 realen SE-Szenarien zeigt praktische Durchführbarkeit und quantifiziert Verbesserungen (+31% Erfolgsrate, -40% Token-Kosten) gegenüber Baselines.
6. **Transferierbarkeit:** Architektur ist nicht auf SE beschränkt. Patterns (Sense-Plan-Act-Reflect, Werkzeugschnittstellen, Sicherheitsschicht) übertragbar auf andere Domänen (DevOps, Data Science, QA).

5.2.1 Wissenschaftlicher Beitrag

Im Kontext der Forschungslandschaft (vgl. Kapitel 2) schließt die Arbeit folgende Lücken:

- Systematisierung agentischer SE-Architekturen (bisher meist ad-hoc Prototypen)
- Fokus auf Produktionsreife (Sicherheit, Kosten, Robustheit) statt nur Aufgabenerfolg
- Transferierbare Design-Patterns statt monolithischer Systeme
- Vergleichbare Evaluation mit reproduzierbaren Benchmarks

5.3 Limitierungen

Trotz der positiven Ergebnisse gibt es folgende Limitierungen, die bei der Interpretation berücksichtigt werden müssen:

5.3.1 Methodische Limitierungen

- **Begrenzte Testmenge:** Die Evaluation auf 25 Tasks (3 Szenarien) liefert belastbare Indikationen, ist aber nicht hinreichend für abschließende Aussagen zur Produktionsreife. Größere und methodisch strengere Evaluationssetups, etwa SWE-bench Pro oder vergleichbare End-to-End-Benchmarks, wären wünschenswert [Ope26; SWE26].
- **Kontrollierte Umgebung:** Experimente erfolgten auf kuratierten Projekten mit sauber definierten Szenarien. Praxiseinsätze weisen häufig ambigere Anforderungen, Altsysteme und unvollständige Dokumentation auf.
- **Skalierungsgrenzen:** Tests beschränkten sich auf Projekte bis 50k LOC. Verhalten bei Millionen LOC (Linux Kernel, Chromium) ist unklar.
- **LLM-Abhängigkeit:** Die Ergebnisse basieren auf Frontier-Modellen der GPT- und Claude-Klasse. Neuere Modellgenerationen können Architektur-Trade-offs verschieben; kleinere oder offene Modelle führen je nach Aufgabe zu abweichenden Kosten-, Latenz- und Qualitätsprofilen.

„Während agentische Systeme Potenzial zeigen, müssen ihre Grenzen in kontrollierten Umgebungen sorgfältig evaluiert werden, bevor sie in Produktionssystemen eingesetzt werden.“ ([Jim+24])

5.3.2 Technische Limitierungen

- **Halluzinationen:** LLMs erzeugen gelegentlich nicht existente APIs oder fehlerhafte Begründungsschritte. Reflexion reduziert dieses Risiko, eliminiert es jedoch nicht.
- **Kontextfenster:** Trotz deutlicher Fortschritte bei langen Kontextfenstern stoßen selbst Modelle mit sehr großen Eingabebudgets bei komplexen Codebasen an praktische Grenzen [Met25]. RAG- und Chunking-Strategien bleiben daher Hilfsmittel, keine vollständige Lösung.
- **Werkzeuglatenz:** Testläufe dauern Sekunden bis Minuten. Bei vielen Werkzeugaufrufen akkumuliert Latenz (Median: 45s für Test-Debugging).
- **Fehlerfortpflanzung:** Frühe Fehler (z. B. falsche Diagnose) propagieren durch iterative Schleifen. Ohne menschliche Aufsicht können Agenten in ineffiziente oder fehlerhafte Suchpfade geraten.

5.3.3 Gesellschaftliche und ethische Limitierungen

Zu berücksichtigen ist auch die gesellschaftliche Dimension. Die verfügbare Literatur deutet darauf hin, dass LLMs und agentische Systeme Tätigkeitsprofile im Software Engineering spürbar verändern können, ohne dass sich daraus bereits lineare oder einheitliche Aussagen über Substitutionseffekte ableiten lassen [Elo+23]. Für wissenschaftliche und industrielle Anwendungen bedeutet dies, dass Produktivitätsgewinne, Qualifikationsverschiebungen und Verantwortungsfragen gemeinsam betrachtet werden müssen.

Weitere ethische Dimensionen:

- **Bias-Verstetigung:** LLMs reproduzieren Biases aus Trainingsdaten. Code-Generierung kann diskriminierende Patterns fortführen.

- **Verantwortlichkeit:** Bei agentengenerierten Fehlern bleibt die Zurechnung zwischen nutzender Organisation, verantwortlicher Fachperson und Modellanbieter eine offene Governance-Frage.
- **Übermäßiges Vertrauen:** Entwicklungsteams könnten kritisches Prüfen reduzieren und Agenten-Outputs zu unkritisch übernehmen – besonders problematisch in sicherheitskritischen Systemen.

5.4 Zukünftige Arbeiten

Auf Basis dieser Arbeit ergeben sich mehrere Richtungen für zukünftige Forschung und Entwicklung:

5.4.1 Kurzfristige Erweiterungen

- **Erweiterte Benchmarks:** Evaluation auf methodisch robusteren Benchmarks wie SWE-bench Pro sowie auf HumanEval-ähnlichen Datensätzen für breitere Validierung
- **Mehrsprachenunterstützung:** Aktuell Python-fokussiert. Erweiterung auf Java, TypeScript, Go für breitere Anwendbarkeit
- **Optimiertes Kontextmanagement:** Hybrid-Strategien (RAG + Summarization + Code-Graph-Navigation) für sehr große Codebasen mit langen Kontextfenstern
- **Fine-Tuning:** Domänenspezifisches Fein-Tuning auf SE-Szenarien könnte Erfolgsraten bei kleineren und günstigeren Modellen verbessern
- **Integration menschlichen Feedbacks:** RLHF-ähnliche Ansätze für kontinuierliches Lernen aus Korrekturen im Entwicklungsteam

5.4.2 Mittel- bis langfristige Forschungsrichtungen

- **Multi-Agenten-Systeme:** Rollenbasierte Kollaboration wie in MetaGPT. Dies könnte Spezialisierung und Parallelisierung verbessern.
- **Kontinuierliches Lernen:** Agenten lernen aus Projekthistorie, Teamkonventionen und Codebasis-Mustern. Episodisches Gedächtnis könnte dabei als Datenquelle dienen.

- **Formale Verifikation:** Integration formaler Methoden (Typprüfung, SMT-Solving, symbolische Ausführung) für sicherheitskritischen Code.
- **IDE-Integration:** Tiefe Integration in Entwicklungsumgebungen mit direktem Arbeitsbereichszugriff und Echtzeitvorschlägen.
- **Hybride Mensch-Agent-Workflows:** Optimale Arbeitsteilung zwischen Automatisierung und fachlicher Kontrolle sowie Werkzeuge für effiziente Delegation und Review.
- **Kosten-Nutzen-Optimierung:** Automatische Entscheidung wann Agent, wann Mensch basierend auf Aufgabenkomplexität, Deadline, Budgetbeschränkungen.

5.4.3 Offene Forschungsfragen

- **Vertrauenswürdigkeit:** Wie lässt sich Vertrauenswürdigkeit messen oder absichern? Welche Rolle können formale Spezifikationen spielen?
- **Emergentes Verhalten:** Bei komplexen Multi-Agenten-Systemen: Wie kontrollieren/verstehen wir emergentes Verhalten?
- **Langzeitgedächtnis:** Wie skalieren semantische/episodische Gedächtnisse über Monate/Jahre? Forgetting vs. Retention Trade-offs?
- **Transfer Learning:** Können Agenten Wissen von Projekt A auf Projekt B transferieren? Wie lässt sich Domänenadaption für neue Codebasen gestalten?

5.5 Schlusswort

Die Arbeit zeigt, dass agentische Architekturen für Software-Engineering-Workflows praktisch umsetzbar sind und unter den gewählten Randbedingungen einen messbaren Nutzen entfalten können. Die entwickelte Referenzarchitektur, Implementierung und Evaluation bilden eine belastbare Grundlage für weiterführende Forschung und praktische Anwendungen.

Zentrale Erkenntnisse:

- **Machbarkeit:** Agenten erreichen 73 % Erfolgsrate bei Refactoring – ein belastbarer Hinweis auf praktische Einsetzbarkeit, jedoch keine hinreichende Evidenz für allgemeine Produktionsreife

- **Effizienz:** 40 % Token-Reduktion durch Optimierung – Effizienzgewinne sind unter geeigneten Randbedingungen erreichbar
- **Sicherheit:** Defense-in-Depth erweist sich als zweckmäßiges Muster – kontinuierliche Überprüfung bleibt jedoch erforderlich
- **Grenzen:** LLM-Halluzinationen, Kontextgrenzen, Latenz bleiben Herausforderungen

Die Technologie ist derzeit nicht hinreichend autonom für eine verantwortbare Vollautomatisierung, kann jedoch bereits als unterstützendes Werkzeug in Entwicklungsprozessen sinnvoll eingesetzt werden. Menschliche Freigabe bleibt sowohl technisch als auch ethisch unverzichtbar.

Zukünftige Arbeiten sollten nicht nur technische Verbesserungen fokussieren, sondern auch soziotechnische Fragen adressieren: Wie verändern Agenten Rollenbilder in der Softwareentwicklung? Wie lassen sich Übergänge in Organisationen verantwortlich gestalten? Und wie kann menschliche Expertise langfristig gesichert werden?

Die Kombination menschlicher Urteilsfähigkeit, Kreativität und Problemlösungskompetenz mit agentischer Automatisierung, Skalierung und Konsistenz kann zu einer verbesserten Unterstützung im Software Engineering beitragen – sofern diese Systeme methodisch sauber evaluiert und verantwortungsvoll eingesetzt werden.

Konkrete Empfehlungen für Praktiker:

- Beginnen Sie mit klar abgegrenzten, gut getesteten Teil-Workflows (z. B. Lint-Fixes, kleine Refactorings) bevor Sie größere Automatisierungsebenen freischalten.
- Implementieren Sie schrittweise menschliche Freigabepunkte für kritische Aktionen und messen Sie kontinuierlich Metriken wie Review-Akzeptanz und Regressionen.
- Nutzen Sie Vektor-basierte Retrieval-Mechanismen für langfristige Projekte, um Kontextkosten zu reduzieren, und automatisieren Sie Summarisierungspipelines für alte Commits.
- Planen Sie regelmäßige Sicherheits-Audits und erweitern Sie Test-Suites um adversariale Prompt-Tests.

Die vorgeschlagenen Schritte erleichtern den verantwortungsvollen Einsatz agentischer Systeme im Alltag und reduzieren Risiken beim Übergang in produktive Umgebungen.

Literaturverzeichnis

- [Cha22] Harrison Chase. *LangChain*. <https://github.com/langchain-ai/langchain>. 2022. (besucht am 12.04.2026).
- [Deu23] Deutsche Forschungsgemeinschaft. *Umgang mit generativen Modellen zur Text- und Bilderstellung in der Wissenschaft und im Förderhandeln der DFG*. Stellungnahme des Präsidiums. 2023. URL: <https://www.dfg.de/resource/blob/289674/230921-stellungnahme-praesidium-ki-ai.pdf> (besucht am 12.04.2026).
- [Elo+23] Tyna Eloundou u. a. „GPTs are GPTs: An Early Look at the Labor Market Impact Potential of Large Language Models“. In: *arXiv preprint arXiv:2303.10130* (2023).
- [Hon+24] Sirui Hong u. a. „MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework“. In: *International Conference on Learning Representations (ICLR)*. 2024.
- [Jim+24] Carlos E. Jimenez u. a. „SWE-bench: Can Language Models Resolve Real-World GitHub Issues?“ In: *International Conference on Learning Representations (ICLR)*. 2024.
- [Kar23] Andrej Karpathy. *Intro to Large Language Models*. <https://karpathy.ai/stateofgpt.pdf>. 2023. (besucht am 12.04.2026).
- [Mar08] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [Met25] Meta. *The Llama 4 Herd: The Beginning of a New Era of Natively Multimodal AI Innovation*. Produktankündigung, abgerufen im April 2026. 2025. URL: <https://ai.meta.com/blog/llama-4-multimodal-intelligence/> (besucht am 12.04.2026).
- [Ope24] OpenAI. *GPT-4 Technical Report*. Techn. Ber. OpenAI, 2024. URL: <https://arxiv.org/abs/2303.08774>.

- [Ope26] OpenAI. *Why SWE-bench Verified no Longer Measures Frontier Coding Capabilities*. Abgerufen im April 2026. 2026. URL: <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/> (besucht am 12.04.2026).
- [Par+23] Joon Sung Park u. a. „Generative Agents: Interactive Simulacra of Human Behavior“. In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 2023.
- [Qin+24] Yujia Qin u. a. „ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs“. In: *International Conference on Learning Representations (ICLR)*. 2024.
- [RF20] Mark Richards und Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020.
- [Sch+23] Timo Schick u. a. „Toolformer: Language Models Can Teach Themselves to Use Tools“. In: *arXiv preprint arXiv:2302.04761* (2023).
- [Shi+23] Noah Shinn u. a. „Reflexion: Language Agents with Verbal Reinforcement Learning“. In: *Advances in Neural Information Processing Systems*. 2023.
- [Som15] Ian Sommerville. *Software Engineering*. 10th. Pearson, 2015.
- [SWE26] SWE-bench. *SWE-bench Leaderboards*. Abgerufen im April 2026. 2026. URL: <https://www.swebench.com/> (besucht am 12.04.2026).
- [Tou+23] Hugo Touvron u. a. „Llama 2: Open Foundation and Fine-Tuned Chat Models“. In: *arXiv preprint arXiv:2307.09288* (2023).
- [Vas+17] Ashish Vaswani u. a. „Attention Is All You Need“. In: *Advances in Neural Information Processing Systems*. 2017.
- [Wan+23] Lei Wang u. a. „A Survey on Large Language Model based Autonomous Agents“. In: *arXiv preprint arXiv:2308.11432* (2023).
- [Wei+22] Jason Wei u. a. „Chain-of-Thought Prompting Elicits Reasoning in Large Language Models“. In: *Advances in Neural Information Processing Systems*. 2022.
- [Wen23] Lilian Weng. *Prompt Engineering*. <https://lilianweng.github.io/posts/2023-03-15-prompt-engineering/>. 2023. (besucht am 12.04.2026).

-
- [Wu+23] Qingyun Wu u. a. „AutoGen: Enabling Next-Gen LLM Applications“. In: *arXiv preprint arXiv:2308.08155* (2023).
- [Xi+23] Zhiheng Xi u. a. „The Rise and Potential of Large Language Model Based Agents: A Survey“. In: *arXiv preprint arXiv:2309.07864* (2023).
- [Yan+24] John Yang u. a. „SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering“. In: *arXiv preprint arXiv:2405.15793* (2024).
- [Yao+23] Shunyu Yao u. a. „ReAct: Synergizing Reasoning and Acting in Language Models“. In: *International Conference on Learning Representations (ICLR)*. 2023.
- [Zha+24] Chunqiu Steven Zhang u. a. „Agentless: Demystifying LLM-based Software Engineering Agents“. In: *arXiv preprint arXiv:2407.01489* (2024).
- [Zhu+23] Mingchen Zhuge u. a. „Mindstorms in Natural Language-Based Societies of Mind“. In: *arXiv preprint arXiv:2305.17066* (2023).

Index

- Agent
 - Zielgerichtetheit, 5
- Agent Controller
 - Implementierung, 26
- Agent Logging, 26
- Agent Loop, 26
- Agent-Transparenz, 37
- Agenten-Architektur, 5
- Agenten-Policies
 - Robustheit, 37
- Agentenarchitektur, 4, 16
 - Entwurf, 15
- Agentic AI, 1
- Agentische Anwendungen, 5
- Architekturdesign, 37
- Architekturprinzipien, 15
- Async/Await, 25
- Asynchrone Verarbeitung, 16
- Audit-Logging, 16
 - Tool-Calls, 38
- Benchmark, 25
- Best Practices, 15
- Chroma Vector-DB, 25
- Chunking, 17
- Circuit Breaker, 16
- Claude, 25
- Code Linting, 25
- Code-Ausführung, 5
- Continuous Testing, 25
- Defense-in-Depth, 38
- Design
 - Architektur-Design, 15
- Distributed Tracing, 25
- Docker
 - Sandboxing, 25
- Empirische Evaluation, 37
- Episodisches Gedächtnis, 16
 - Design, 37
- Error-Handling, 16
 - Agent, 26
- Evaluation, 2, 25
 - Agenten, 37
- Event-Logging, 25
- Externe Tools, 5
- Fallback-Mechanismus, 25
- Few-Shot-Learning, 5
- File-Access-Kontrolle, 38
- GPT-4, 25
- Implementierung, 25
- Implementierungspraxis, 25
- In-Context-Learning, 5
- Input-Validation
 - Tool-Calls, 38
- Iterative Entwicklung, 25
- Kontext-Persistierung, 37
- Kontextpersistierung, 6

- LangChain, 25
- Langkontext, 5
- Large Language Models
 - Grundlagen, 5
- Least Privilege, 16
- Least-Privilege, 38
- LLM, 16
- LLM-gesteuerte Policy, 6
- Logging, 16
- Modularität, 16
- Nachvollziehbarkeit, 37
- Parallelisierung, 16
- Permissions-System, 38
- Planung
 - Agent-Planung, 15
 - Implementierung, 26
 - in Policies, 37
- Policy
 - Entscheidungsfindung, 6
- Policy-Engine, 15
- Policy-Modellierung, 37
- Prometheus, 25
- Prompt-Injection, 38
- Python, 25
- RAG
 - Memory, 16
- ReAct, 16
 - Anwendung, 37
- Referenzarchitektur, 37
- Reflexion, 26
- Reflexionsmechanismen, 37
- Regelbasierte Policy, 6
- Reproduzierbarkeit, 16
- Retry, 16
- Sandbox
 - Docker, 38
- Sandboxing, 16
- Schema-Validation, 37
- SE-Workflows, 37
- Selbstbewertung, 6
- Sense-Plan-Act-Reflect, 15
- Sicherheit, 15
 - Architektur, 37
- Sicherheits-Audits, 38
- Software Engineering, 1
- Standardisierung, 16
- State Management, 6
- Strukturierte Policies, 37
- Summarization, 17
- Task-Dekomposition, 6
- Textkorpora
 - Training, 5
- Thought-Action-Observation, 37
- Timeout, 16
- Token-Optimierung, 17
- Tool-Discovery, 16
- Tool-Integration, 15
 - Sicherheit, 37
- Tool-Interfaces
 - Strukturierung, 37
- Tool-Metadaten, 16
- Tool-Nutzung, 5
- Tool-Status, 6
- Tool-Wrapper, 16
- Transformer

Architektur, 5	Websuche, 6
Type Hints, 25	
VCS-Operationen, 6	Zielerreichung, 5
Vektor-Datenbank, 16	Zustandsaktualisierungen, 6

A Verzeichnis der KI-Nutzung

Erklärung zur Nutzung KI-basierter Werkzeuge

Die Erstellung der vorliegenden Arbeit wurde durch den Einsatz KI-basierter Werkzeuge unterstützt. Eine detaillierte tabellarische Übersicht der verwendeten Systeme sowie des jeweiligen Zwecks und Umfangs ihrer Nutzung findet sich in Tabelle A.1.

Alle von KI-Systemen generierten Vorschläge und Inhalte wurden von mir eigenständig geprüft, fachlich bewertet und bei Bedarf überarbeitet oder verworfen. Die Verantwortung für die Auswahl, inhaltliche Einordnung, Interpretation sowie die endgültige Fassung des Textes liegt vollständig bei mir.

Grundlage für den Umgang mit den Werkzeugen bilden die aktuellen Empfehlungen der Deutschen Forschungsgemeinschaft (DFG) zu KI in wissenschaftlichen Arbeiten [Deu23].

Declaration on the Use of AI-based Tools

The preparation of this thesis was supported by the use of AI-based tools. A detailed tabular overview of the systems used, as well as the respective purpose and scope of their use, is provided in Table A.1.

All suggestions and content generated by AI systems were independently reviewed, professionally evaluated, and revised or rejected by me where necessary. I assume full responsibility for the selection, categorization, interpretation, and the final version of the text.

The handling of these tools follows the current recommendations of the German Research Foundation (DFG) for scientific work [Deu23].

Tabelle A.1 listet alle in dieser Arbeit verwendeten KI-Werkzeuge auf.

Tabelle A.1: Verzeichnis der in dieser Arbeit genutzten KI-Werkzeuge

KI-Tool	Zweck	Kontext/Eingangstext	Generierte Ausgabe	Kapitel
Claude Opus 4.5	Konzeptualisierung	Strukturierung des methodischen Ansatzes	Vorschlag zur Gliederung der Methodik	Kap. 3.1
Claude Opus 4.5	Code-Refactoring	Optimierung bestehender Algorithmen	Vorschläge zur Effizienzverbesserung	Kap. 4.3
Cohere Command A	RAG/Agentik	Entwurf eines Retrieval-Flows	Vorschlag für Tool-Aufrufe und Prompt-Struktur	Kap. 3.2
GitHub Copilot	Code-Vervollständigung	Python-Funktion für Datenverarbeitung	Vorschläge für Funktionsimplementierung	Kap. 4.2
Gemini 3 Pro	Literaturrecherche	Zusammenfassung aktueller Forschungsarbeiten	Überblick über Stand der Technik	Kap. 2.1
Mistral Large 3	Technische Zusammenfassung	Auswertung von API-/RFC-Dokumenten	Kompakte Zusammenfassung der Kernaussagen	Kap. 2.4
OpenAI GPT-5.2	Formulierungsverbesserung	Überarbeitung der Einleitung	Alternative Formulierungen zur Problemstellung	Kap. 1.2
xAI Grok 4.1	Agentische Aufgaben	Recherche zu aktuellen Statistiken	Stichpunkt-Zusammenfassung mit Quellenhinweisen	Kap. 2.3

Erklärung der Selbstständigkeit

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben.

Frankfurt am Main, den 12. April 2026

Maria Musterfrau